

✅ DAY 3 (MODIFIED FINAL VERSION)

👉 Goal:

PDF → Page-wise structured content

(Text + Tables + Page info)

✅ FILES

app/parser/

└─ pymupdf_parser.py

└─ pdfplumber_table.py

└─ page_builder.py ✅ NEW

app/ingestion/

└─ pipeline.py

✅ app/parser/pymupdf_parser.py

```
import fitz
```

```
def parse_pymupdf(file_path):
```

```
    doc = fitz.open(file_path)
```

```
    pages = []
```

```
    for i in range(doc.page_count):
```

```
        page = doc.load_page(i)
```

```
        text_lines = str(
```

```
            page.get_text("text") or ""
```

```
        ).splitlines()
```

```
cleaned = []

for line in text_lines:
    line = line.strip()

    if line:
        cleaned.append(line)

pages.append({
    "page": i,
    "text": cleaned
})

return pages
```

✅ **app/parser/pdfplumber_table.py**

```
import pdfplumber
```

```
def clean_row(row):
```

```
    return [
        cell.strip()
        for cell in row
        if cell not in (None, "", " ")
    ]
```

```
def extract_tables_pdf(file_path):
```

```
    tables = []
```

```
with pdfplumber.open(file_path) as pdf:
```

```
    for page_num, page in enumerate(pdf.pages):
```

```
        raw_tables = page.extract_tables()
```

```
        if not raw_tables:
```

```
            continue
```

```
        for table_id, table in enumerate(raw_tables):
```

```
            cleaned_table = []
```

```
            for row in table:
```

```
                cleaned = clean_row(row)
```

```
                if len(cleaned) >= 2:
```

```
                    cleaned_table.append(cleaned)
```

```
            if len(cleaned_table) < 2:
```

```
                continue
```

```
            tables.append({
```

```
                "page": page_num,
```

```
                "table_id": f"page_{page_num}table{table_id}",
```

```
                "data": cleaned_table
```

```
            })
```

```
print(f"✅ Table extracted from page {page_num}")
```

return tables

✓ **app/parser/page_builder.py**

```
from app.parser.pymupdf_parser import parse_pymupdf
from app.parser.pdfplumber_table import extract_tables_pdf
```

```
def build_page_view(file_path):
```

```
    parsed_pages = parse_pymupdf(file_path)
```

```
    extracted_tables = extract_tables_pdf(file_path)
```

```
    table_map = {}
```

```
    for t in extracted_tables:
```

```
        page = t["page"]
```

```
        if page not in table_map:
```

```
            table_map[page] = []
```

```
            table_map[page].append(t)
```

```
    final_pages = []
```

```
    for p in parsed_pages:
```

```
        page_num = p["page"]
```

```
final_pages.append({
    "page": page_num,
    "text": p["text"],
    "tables": table_map.get(page_num, [])
})
```

```
return final_pages
```

✓ **app/ingestion/pipeline.py**

```
from app.parser.page_builder import build_page_view
```

```
def process_pdf(file_path):
```

```
    pages = build_page_view(file_path)
```

```
    return {
```

```
        "pages": pages
```

```
    }
```

✓ **tests/test_ingestion.py**

```
from app.ingestion.pipeline import process_pdf
```

```
result = process_pdf("data/sample.pdf")
```

```
for page in result["pages"]:
```

```
    print(f"\n===== PAGE {page['page']} =====\n")
```

```
    print("TEXT:\n")
```

```
for line in page["text"][:10]:
    print(line)

print("\nTABLES:\n")

for table in page["tables"]:

    print("TABLE ID:", table["table_id"])

    for row in table["data"][:10]:
        print(row)

    print("-----")
```

OUTPUT YOU SHOULD GET

===== PAGE 0 =====

TEXT:

TPDS

TEST REPORT

Objective:

...

TABLES:

TABLE ID: page_0_table_0

['P201 BRAKE TEMPERATURE MAPPING', 'CDT', 'GHD', 'HSD']

['Distance Covered (km)', '149', '243', '264']

['Front pad peak temperature', '163', '179', '188']

✅ WHAT DAY 3 ACHIEVES

✓ page-wise structure

✓ table-wise structure

✓ table_id

✓ page tracking

✓ clean extraction

✓ ready for normalization

✅ NEXT = DAY 4

👉 Day 4 will convert this:

['Front pad peak temperature', '163', '179', '188']

into:

```
{  
  "metric": "Front pad peak temperature",  
  "CDT": 163,  
  "GHD": 179,  
  "HSD": 188  
}
```

and also:

table_type

source_pdf

metadata

normalized columns

